

“제자리” 문제 풀이

작성자: 윤교준

부분문제 1

$A_1 = 1$ 이면 답은 0이고, $A_1 \neq 1$ 이면 답은 1이다.

부분문제 2

제거할 카드를 정한 후, 남은 카드가 제자리 상태가 되는지 판별한다.

제거할 카드를 선택하는 가짓수가 $\mathcal{O}(2^N)$ 이고, 남은 카드가 제자리 상태인지 $\mathcal{O}(N)$ 에 판별할 수 있다.

따라서, 전체 문제를 $\mathcal{O}(N \times 2^N)$ 에 해결할 수 있다.

부분문제 3

제자리 상태의 카드의 값은 차례대로 $1, 2, 3, \dots, K$ 의 값을 가져야 한다.

남은 카드가 제자리 상태임을 가정하고, K 의 값을 정한 후, 이것이 가능한지 판별하자. 가능한 K 중 최댓값을 $K_{\max}$ 라 하면, 문제의 답은 $N - K_{\max}$ 이다.

첫 번째 카드부터 차례대로 확인하면서, 수열 $1, 2, 3, \dots, K$ 의 prefix 중 어디까지 만들 수 있는지 탐욕적으로 확인할 수 있다.

가능한 K 의 값은 $\mathcal{O}(N)$ 가지가 있고, 고정된 K 에 대하여 가능 여부를 $\mathcal{O}(N)$ 에 판별할 수 있다.

따라서, 전체 문제를 $\mathcal{O}(N^2)$ 에 해결할 수 있다.

부분문제 4

부분문제 3의 아이디어를 응용하자.

K 의 값을 고정하지 않아도 충분하다.

수열 $1, 2, 3, \dots, N$ 의 prefix 중 어디까지 만들 수 있는지, 첫 번째 카드부터 차례대로 확인하면서 탐욕적인 전략을 택해도 정당하다.

따라서, 전체 문제를 $\mathcal{O}(N)$ 에 해결할 수 있다.

추가적으로, $\mathcal{O}(1)$ 의 공간 복잡도로 문제를 해결할 수 있다.

“카드 바꾸기” 문제 풀이

작성자: 박선재

부분문제 1

N 이 3 이하이다. N 이 2 이상이므로, $N = 2$ 일 때와 $N = 3$ 일 때로 나누어 생각할 수 있다.

우선 $N = 2$ 이면 바꿀 필요가 없다. 따라서 $N = 2$ 일 경우 답은 0이다. $N = 3$ 일 경우, 카드에 적혀있는 수가 왼쪽부터 순서대로 a, b, c 라고 했을 때, $c - b = b - a$ 이면, 바꿀 필요가 없다. 이때 답은 0이다. $c - b \neq b - a$ 이면, $d = b - a$ 라고 했을 때, 마지막 수를 $b + d$ 로 바꾸면 되므로, 답이 1이 된다. 따라서 경우를 나누어 문제를 해결할 수 있다. 시간 복잡도는 $O(1)$ 이다. 제한 시간 내에 충분히 해결할 수 있다.

부분문제 2

먼저, 수들을 적절히 바꾼 후 최종상태에서 카드에 적혀있는 수들은 적절한 정수 a, d 에 대해 왼쪽부터 순서대로 $a, a + d, \dots, a + (N - 1)d$ 가 돼야 한다.

$N \leq 5$ 인 경우, 원래 카드 중 2개 이상의 카드에 적혀 있는 수들은 바뀌지 않으므로, 선택할 수 있는 모든 카드 쌍에 대해 그 카드들에 적혀있는 수를 바꾸지 않을 수 있는지 판단한다. 가능하다면, 위의 a, d 가 정해지므로, 적혀있는 수를 바꿔야 하는 카드의 개수를 세어준다. 그 후, 각 경우에서 적혀있는 수를 바꿔야 하는 카드 개수 중 최솟값을 답으로 하면 된다.

$N > 5$ 인 경우, 인접한 두 카드에 적힌 수를 모두 바꾸지 않아도 되는 경우가 반드시 존재한다. 그래서 모든 인접한 카드 쌍에 대해 그 카드들에 적혀있는 수를 바꾸지 않는 경우에 대해 $N \leq 5$ 인 경우와 비슷하게 답을 계산해주면 된다.

$O(N)$ 개 이하의 경우에 대해 바꿔야 하는 카드의 개수를 세는 데 $O(N)$ 의 시간이 걸린다. 따라서, 전체 시간복잡도는 $O(N^2)$ 이다. 제한 시간 내에 충분히 해결할 수 있다.

부분문제 3

부분문제 2의 풀이에서 설명했듯이, 수들을 적절히 바꾼 후 최종상태에서 카드에 적혀있는 수들은 적절한 정수 a, d 에 대해 왼쪽부터 순서대로 $a, a + d, \dots, a + (N - 1)d$ 가 돼야 한다.

그리고, 원래 카드에 적혀 있던 수 중에 적어도 하나는 바꾸지 않아도 된다는 점을 이용한다. i 번째 ($1 \leq i \leq N$) 카드를 바꾸지 않고, d 값이 정해진다면, $a = x - (i - 1)d$ 로 같이 정해지므로, 몇 개의 카드에 적혀있는 수들을 바꿔야 하는지 계산할 수 있다. 따라서 왼쪽부터 모든 카드를 순서대로 순회하면서, 가능한 모든 d 값에 대해 적혀있는 수를 바꿔야 하는 카드의 개수 중 최솟값을 계산해 주면 된다.

각 카드에 대해서, 시도 가능한 d 값의 개수가 $O(d)$ 이고, 각 경우에 대해서 적혀있는 수를 바꿔야 하는 카드의 개수를 세는 데 $O(N)$ 의 시간이 걸린다. 따라서 전체 시간복잡도는 $O(N^2d)$ 가 된다. d 의 범위가 작으므로, 제한 시간 내에 해결할 수 있다.

부분문제 4

먼저 답이 아무리 커도 $N - 2$ 이하임을 알 수 있다. 이는 왼쪽에서부터 첫 번째, 두 번째 카드에 적힌 수를 각각 a, b 라고 할 때, $d = b - a$ 로 두면 첫 번째, 두 번째 카드에 적힌 수는 바꿀 필요가 없다. 따라서 이 경우에 적혀있는 수를 바꿔야 하는 카드의 개수가 $N - 2$ 임을 알 수 있다. 그러므로 $N - 2$ 개 이하의 카드만 바꿔도 원하는 형태로 만들 수 있다.

답이 $N - 2$ 이하라는 것은, 적혀있는 수를 바꾸지 않아도 되는 카드가 적어도 2개는 있다는 것이다. 그러므로 모든 카드 쌍에 대해, 현재 보고 있는 카드가 왼쪽에서 i, j 번째 카드이고, ($i < j$) 적혀있는 수가 순서대로 x_i, x_j 라고 하자.

i, j 번째 카드를 바꾸지 않아도 되는 경우에 대해서 생각하면, 이때 $d = \frac{x_j - x_i}{j - i}$ 로 정해진다. d 가 정해졌으므로, $a = x_i - (i - 1)d$ 로 정해진다. a, d 가 정해졌으므로, 이 경우에 바뀌어야 할 카드의 개수를 정할 수 있다. 단, d 가 정수여야 하므로 $x_j - x_i$ 가 $j - i$ 의 배수여야 한다. 그렇지 않으면 불가능하다. 이 경우에는 넘어간다. 모든 경우에 대해 적혀있는 수를 바꿔야 하는 카드의 개수 중 최솟값을 답으로 하면 된다.

가능한 카드 쌍의 개수가 $\frac{N(N-1)}{2}$ 개이고, 각 경우에 대해 적혀있는 수를 바꿔야 하는 카드의 개수를 세는데 $O(N)$ 의 시간이 걸리므로, 전체 시간복잡도는 $O(N^3)$ 이 된다. 제한 시간 내에 전체 문제를 해결할 수 있다.

“트리와 쿼리” 문제 풀이

작성자: 윤교준

부분문제 1

가능한 트리의 형태가 세 가지밖에 없으므로, 조건문으로 쉽게 해결할 수 있다.

부분문제 2

S 에 속한 모든 두 정점쌍에 대하여 S 위에서 연결되어 있는지 여부를 판별하자.

두 정점이 S 위에서 연결되어 있는지는 DFS 등을 이용하면 $\mathcal{O}(N)$ 에 판별할 수 있다.

따라서, 각 쿼리당 $\mathcal{O}(NK^2)$ 에 문제를 해결할 수 있다.

부분문제 3

트리에서 S 에 속한 정점만 남기고 다른 모든 정점을 제거하면, 여러 개의 트리로 이루어진 포레스트(Forest)가 된다.

이 포레스트의 연결 컴포넌트를 “ S 위에서 연결 컴포넌트”라고 하자.

S 의 연결 강도는 S 위에서 연결 컴포넌트 각각의 크기를 통하여 계산할 수 있다.

따라서, S 에 속하는 정점만을 사용하여 DFS를 수행하면, 각 쿼리당 $\mathcal{O}(N)$ 에 문제를 해결할 수 있다.

일반적으로, 정점 집합의 부분 집합 $V' \subseteq V$ 를 방문하는 DFS의 시간 복잡도는 다음과 같다:

$$\sum_{v \in V'} \deg(v)$$

V' 에 속하는 각 정점 v 마다 $\deg(v)$ 개의 간선을 참조하기 때문이다.

이 문제의 경우, 일반적인 DFS를 수행한다면, 시간 복잡도는 다음과 같다:

$$\sum_{v \in S} \deg(v)$$

이 값은 집합의 크기 $|S| = K$ 보다 유의미하게 클 수 있음에 유의하라.

예를 들어, 1번 정점에 다른 모든 정점이 붙어 있는 트리 위에서 $S = \{1, 2, \dots, K\}$ 를 생각하자. $\deg(1) = N - 1$ 이므로 K 의 값에 무관하게, degree의 합이 $\mathcal{O}(N)$ 임을 확인할 수 있다.

부분문제 4

Union-find 기법을 응용하자.

입력에서 주어진 트리에서 아무 정점이나 잡아 루트로 만들자.

S 에 속하는 각 정점 v 마다, v 의 부모 정점이 S 에 속하는 경우에만 v 와 v 의 부모 정점을 union하자.

Union된 컴포넌트의 각 크기를 통하여 S 의 연결 강도를 계산할 수 있다.

부분문제 3과 다르게, S 에 속하는 각 정점 v 에 대하여 $\deg(v)$ 개의 간선이 아닌, 단 하나의 간선만을 참조함에 유의하라.

따라서, 각 쿼리당 $\mathcal{O}(K\alpha(N))$ 에 문제를 해결할 수 있다.

Union-find를 사용하지 않고 각 쿼리마다 정점을 깊이 순으로 정렬한 후, 깊은 정점부터 컴포넌트의 크기를 계산해 나가면서 $\mathcal{O}(N + \sum_i K_i \log K_i)$ 에 문제를 해결할 수도 있다. 쿼리를 오프라인으로 받을 경우, 계수 정렬을 사용할 수 있어서 위 알고리즘의 복잡도가 $\mathcal{O}(N + \sum_i K_i)$ 로 최적화된다.

“빨강 파랑” 문제 풀이

작성자: 김동현

$[a, b] \times [c, d]$ 상자란, 왼쪽 아래 꼭짓점이 (a, b) 이고 오른쪽 위 꼭짓점이 (c, d) 인 각 변이 좌표축에 평행한 직사각형(내부 및 경계 모두 포함)을 의미한다.

부분문제 1

W, H 및 모든 좌표의 값이 1 이상 50 이하이므로, 직사각형의 왼쪽 꼭짓점이 $[-50, 50] \times [-50, 50]$ 상자 밖에 있으면 점을 하나도 포함하지 못하고, 따라서 고려할 필요가 없음을 알 수 있다.

직사각형의 왼쪽 꼭짓점 좌표를 고정하면 그 직사각형이 빨간 점과 파란 점을 각각 몇 개 포함하는지를 $O(N + M)$ 시간에 구할 수 있다.

이제 이 범위 내의 모든 가능한 정수 좌표에 직사각형의 왼쪽 꼭짓점을 놓는 경우를 다 시도해 보면 $O(N + M)$ 짜리 과정을 10000번 시도하므로 제한 시간 내에 문제를 해결할 수 있다.

부분문제 2

좌표의 값이 1 이상 1000 이하이므로, 직사각형의 왼쪽 꼭짓점이 $[-1000, 1000] \times [-1000, 1000]$ 상자 내에 있는 경우만 고려하면 된다. 대신, 이제는 각 좌표마다 해당하는 직사각형이 포함하는 빨간 점 개수와 파란 점 개수의 차를 빠르게 구할 수 있어야 한다.

(x, y) 좌표에 위치한 점을 포함하는 직사각형은 왼쪽 아래 꼭짓점 좌표를 (a, b) 라고 할 때 $x - W \leq a \leq x$, $y - H \leq b \leq y$ 를 만족해야 한다. 즉, (a, b) 가 $[x - W, x] \times [y - H, y]$ 상자에 속해야 한다.

편의상 모든 좌표에 1000을 우선 더했다고 하고, 2001×2001 크기의 2차원 배열이 있다고 하자. 입력의 각 점 (x, y) 에 대해 배열의 $[x - W, x] \times [y - H, y]$ 상자 부분에 (그 점이 빨간색일 경우 1, 파란색일 경우 -1)을 더했다고 하자. 그렇다면 문제의 답은 배열에 들어있는 값들 중 절댓값의 최댓값, 즉 $\max(\text{최댓값}, -\text{최솟값})$ 이 된다. 정답에 해당하는 값이 적혀있는 위치의 인덱스에서 각각 1000을 다시 빼면 직사각형 왼쪽 아래 꼭짓점의 좌표 역시 얻을 수 있다.

특정 직사각형 위치의 Cell들에 일정한 값을 더하고 최댓값 및 최솟값을 구하는 것을 서브태스크 제약 조건 하에서 빠르게 처리하기 위해서는 크게 두 가지 방법을 사용할 수 있다.

첫 번째 방법은 일명 "2차원 누적합" 이라고 부르는, 아래와 같은 알고리즘을 사용하는 것이다. 배열이 $[0, R - 1] \times [0, C - 1]$ 상자라고 하자.

- $[a, b] \times [c, d]$ 상자에 값 v 를 더할 때에는 일단 다음의 4가지 연산을 수행한다.

```
arr[a][b] += v; arr[a][d + 1] -= v; arr[c + 1][b] -= v; arr[c + 1][d + 1] += v;
```

- 모든 더하기 연산이 끝난 뒤, 마지막으로 다음을 수행한다.

```
for i in [0, R - 1]: for j in [1, C - 1]: arr[i][j] += arr[i][j - 1];
```

```
for i in [1, R - 1]: for j in [0, C - 1]: arr[i][j] += arr[i - 1][j];
```

더하기 연산 하나당 $O(1)$, 마지막 처리는 $O(RC)$ 이다. 이후 배열의 값은 단순 참조로 $O(1)$ 에 알 수 있다. 모든 더하기 연산이 끝난 뒤에야 마지막 처리를 하고 각 배열 요소의 값을 알 수 있음에 유의하라.

더하기 연산은 $O(N + M)$ 번 수행하고 배열의 크기가 2001×2001 이므로 제한시간 안에 문제를 풀 수 있다.

실제 구현 시에는 Out-of-Bound error를 예방하기 위해 배열 각 차원의 크기를 조금 더 여유있게 할당하는 편이 좋다.

두 번째 방법은 Plane Sweeping 기법을 사용하는 것이다. Plane Sweeping이란, 주로 2차원 공간에서 특정 축을 기준으로 잡고 직선을 그 축에 대해 한 방향으로 쭉 움직이면서 직선 위에서 나타나는 변화를 효율적으로 관리하는 방식의 기법을 총칭한다.

2차원 배열 위에서 이를 다른 말로 표현하면 $\text{row}[i] = \{\text{arr}[i][0], \text{arr}[i][1], \dots, \text{arr}[i][C-1]\}$ 이라고 할 때 k 를 0부터 $(R-1)$ 까지 하나씩 늘려가면서 row 배열에 나타나는 변화를 관리하는 것이다.

$[a, b] \times [c, d]$ 상자에 값 v 를 더한다고 할 때, 이는 다음 2개의 변화로 설명이 가능하다.

- $k = a$ 가 되는 순간 $\text{row}[k]$ 배열의 $[c, d]$ 구간에 v 를 더한다.
- $k = b+1$ 이 되는 순간 $\text{row}[k]$ 배열의 $[c, d]$ 구간에 v 를 뺀다.

가장 단순하게 이를 구현하면, $\text{row}[k]$ 배열을 단순 1차원 배열로 관리하고, 변화가 일어날 때마다 해당하는 Cell에 일일이 값을 더하는 방식이 될 것이다. 최댓값 및 최솟값의 위치는 $k = 0, 1, \dots, (R-1)$ 에 대해 $\text{row}[k]$ 배열의 값을 모두 확인하면 알 수 있다.

값을 더하는 상자가 $N + M$ 개 있으므로 변화는 총 $2(N + M)$ 개가 있고, 하나의 변화를 적용하는 데 $O(C)$ 만큼의 시간이 걸리고, 모든 Cell을 한 번씩 확인하는 데 $O(RC)$ 만큼 걸리므로 총 시간복잡도가 $O((R + N + M)C)$ 가 되어 이 방식으로든 제한시간 내에 문제를 해결할 수 있다.

부분문제 3

직사각형의 왼쪽 아래 꼭짓점의 후보를 줄일 수 있다. 만약 직사각형의 경계에 점이 놓이지 않았다면, 경계에 점이 하나 이상 놓일 때까지 직사각형을 적절히 "당겨도" 포함하는 점의 집합이 변하지 않기 때문이다.

구체적으로 따져 보면, 직사각형의 왼쪽 아래 꼭짓점 (a, b) 가 다음 조건을 만족하는 경우만 보아도 충분하다.

- 입력에서 주어진 어떤 점 (x_i, y_i) 에 대해 $a = x_i + 1$ 또는 $a = x_i - W$ 를 만족
- 입력에서 주어진 어떤 점 (x_j, y_j) 에 대해 $b = y_j + 1$ 또는 $b = y_j - H$ 를 만족

x 좌표와 y 좌표의 후보가 각각 $O(N + M)$ 개 있으므로, 가능한 좌표는 총 $O((N + M)^2)$ 가지가 있다. 좌표를 하나 고정하면 $O(N + M)$ 시간에 빨간 점과 파란 점 개수의 차를 구할 수 있으므로 $O((N + M)^3)$ 시간복잡도로 문제가 해결된다.

부분문제 4

부분문제 2와 3을 푸는 데 쓰인 아이디어들을 결합하면 부분문제 4를 해결할 수 있다.

부분문제 3을 풀 때 x, y 좌표의 후보가 각각 최대 $2(N + M)$ 개 뿐임을 관찰하였다. 이 후보 x, y 좌표들을 각각 $\{x_0, x_1, \dots, x_{R-1}\}, \{y_0, y_1, \dots, y_{C-1}\}$ 이라고 하자. 이제 다른 좌표값들은 중요하지 않으므로 (x_i, y_j)

좌표를 $R \times C$ 2차원 배열의 (R, C) Cell에 대응시키면 $R, C \leq 4000$ 이 성립하게 된다. 이제 부분문제 2를 풀 때 사용하였던 방법을 마찬가지로 적용할 수 있다.

좌표 압축 이후에는 직사각형의 높이 및 너비가 더 이상 일정하지 않음에 유의하라.

부분문제 5

부분문제 2, 4의 풀이에 쓰인 두 가지 방법 중 Plane Sweeping 기법을 자료구조를 통해 최적화할 수 있다. 구체적으로, Plane Sweeping 과정에서 필요한 다음의 2가지 연산을 빠르게 지원하는 자료구조가 필요하다.

- 특정 구간의 Cell에 일정한 값 더하기
- 최댓값(또는 최솟값)이 적힌 Cell의 위치 구하기

이는 Lazy Propagation을 사용한 Segment Tree로 구현할 수 있다. 이 경우 두 가지 연산을 모두 $O(\log(N + M))$ 에 처리할 수 있으므로 전체 문제를 $O((N + M) \log(N + M))$ 에 해결할 수 있다.